La web del terror accesible

Auditoría de calidad (ISO/IEC 25010), accesibilidad (WCAG 2.1 AA) y testing

Proyecto: Concurso de disfraces de Halloween (API REST con PHP, PDO y MySQL)

Integrantes: Polcowñuk Matias, Grasiozetti Luciano

Materia: Paradigmas y Lenguajes de Programación III

Profesor: Sicardi Dante

1. Objetivo

Convertir el proyecto de Halloween en un producto accesible, seguro y normado: aplicar WCAG 2.1 nivel AA, verificar la navegación por teclado y auditar el sistema con la matriz ISO/IEC 25010.

2. Medidas de accesibilidad aplicadas (WCAG 2.1)

Criterio WCAG	Implementación en el código
1.1.1 Contenido no textual	Cada foto de disfraz tiene alt descriptivo: «Foto del disfraz [nombre]».
1.3.1 Información y relaciones	Landmarks header, nav, main y footer; un h1 por página; tabla del admin con caption y th scope=col.
1.4.3 Contraste (mínimo)	Paleta con contraste mayor a 4.5:1 (texto claro sobre violeta oscuro, naranja sobre fondo oscuro).
2.1.1 Teclado	Todo es nativo: button, a, input. Sin div clicables.
2.4.1 Saltar bloques	Enlace «Saltar al contenido» al inicio de cada página.
2.4.2 Título de página	Un title distinto por página.
2.4.7 Foco visible	Regla :focus-visible con contorno amarillo de 3 px.
3.3.2 Etiquetas o instrucciones	label for en cada campo; ayuda de contraseña con aria-describedby.
4.1.3 Mensajes de estado	Contenedor con role=status y aria-live=polite; texto inyectado con .textContent.

3. Matriz de auditoría ISO/IEC 25010

Característica	Evidencia técnica	Estado
1. Adecuación funcional	El backend PHP procesa y persiste datos en MySQL vía PDO: registro (registro.php), login, voto con transacción (votar.php), CRUD de disfraces (crear/editar/eliminar.php). Votos duplicados bloqueados con UNIQUE (id_usuario, id_disfraz).	Cumple
2. Eficiencia de desempeño	listar.php devuelve solo id, nombre, descripción y votos, con la URL de la imagen (no el BLOB). imagen.php sirve la foto con Cache-Control. Consultas simples y con índices (PK y UNIQUE).	Cumple
3. Compatibilidad	Arquitectura desacoplada: public/ (HTML, CSS y JS vanilla con fetch) y api/ (PHP) se comunican solo por JSON.	Cumple
4. Fiabilidad	Cada endpoint usa try/catch (PDOException), registra el error con error_log sin exponerlo al cliente y devuelve códigos semánticos: 200, 201, 400, 401, 403, 404, 405, 409 y 500. Rollback en la transacción de voto.	Cumple
5. Seguridad (OWASP)	password_hash(PASSWORD_BCRYPT) y password_verify; prepare() en todas las consultas (previene SQLi); session_regenerate_id al iniciar sesión; cookies HttpOnly y SameSite; rol admin validado en el servidor (requerirAdmin); imágenes validadas por MIME real y tamaño; salida al DOM con textContent (previene XSS).	Cumple
6. Usabilidad / Accesibilidad	lang=es, title por página, landmarks, un h1 por página, label for en cada campo, alt en imágenes, aria-label único en botones, aria-live=polite con textContent, :focus-visible, enlace Saltar al contenido, contraste AA, objetivos de 44 px.	Cumple
7. Mantenibilidad	Capas separadas: config/, includes/, api/, public/, database/. Cada archivo tiene una sola responsabilidad; helpers compartidos.	Cumple

4. Evidencia de Lighthouse

Ejecutar Lighthouse (F12, pestaña Lighthouse, categoría Accessibility) en cada página y completar la tabla. Insertar las capturas debajo.

Página	Puntaje después
index.html	100
login.html	100
registro.html	100
subir.html	100
admin.html (con sesión admin)	100

